01 / HOW TO USE THIS HANDBOOK

Project handbook

Architecture, working behavior and the reasons behind the proposed Agentic AI Governance Framework.

Living design baseline - v0.1

This handbook documents the intended system as of 09 October 2026. All components, endpoints, policies and procedures below are proposed. No implementation or test evidence is available yet. As the project is built, update each section with its actual behavior, tested version, limitations and evidence.

The system in plain language

An agent is software that can choose steps and use tools to complete a task. Our framework gives that software a registered identity and approved purpose, checks its authority before it acts and keeps a record of the decision and result. A human owner remains accountable for how the agent is configured and used.

Think of three linked questions: Which agent is this? Is this specific action allowed for this task? What actually happened? The registry and identity service answer the first; the gateway and policy engine answer the second; the audit timeline answers the third.

Read this section	What you will understand
2. Architecture and boundaries	Where controls run and why agents cannot authorize themselves.
3. A complete worked example	How one request moves through identity, delegation, tools, approval and evidence.
4. Identity and lifecycle	How agents are registered, activated, suspended and retired.
5. Policy and data controls	How access decisions and sensitive-field restrictions work.
6. Records and relationships	What the framework stores and how records connect.
7. Proposed interfaces	The API boundaries and example decision/audit contracts.
8. Operations and troubleshooting	How to respond to failures and change policies safely.
9. Design decisions and risks	Why this approach was chosen and what remains unresolved.
10. Maintenance and glossary	How to keep the handbook accurate through implementation.

Companion documents

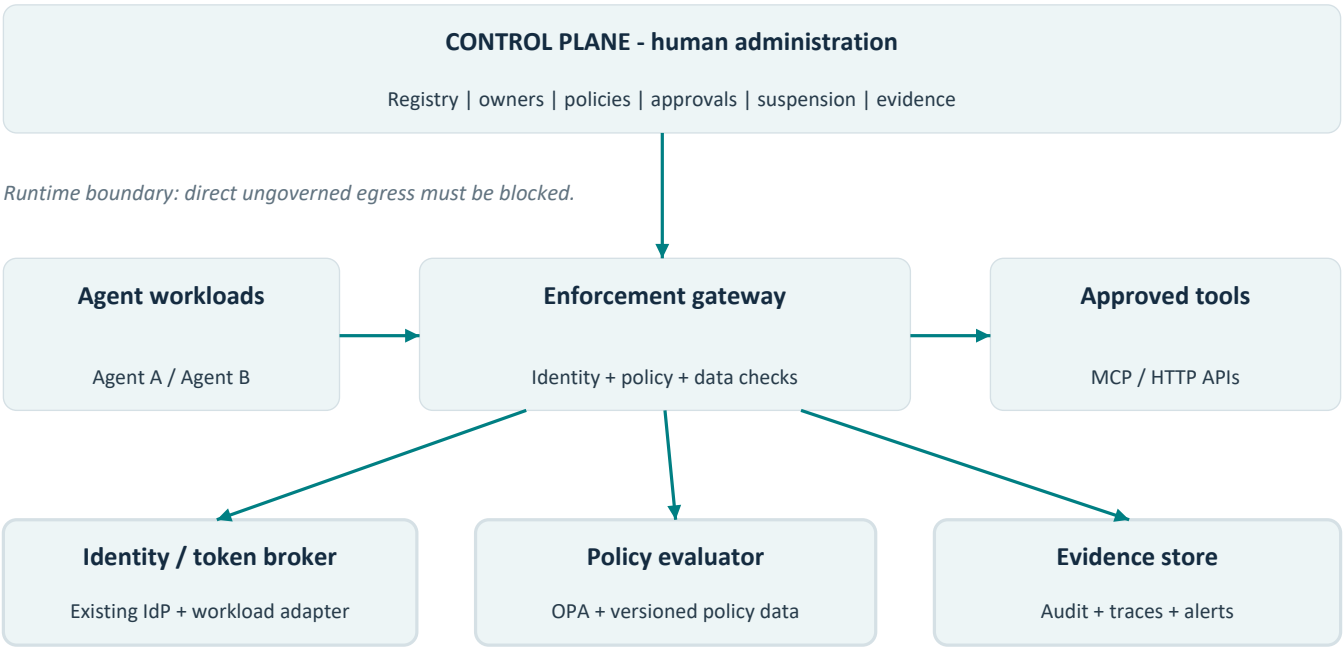
The problem statement records requirements R01-R12 and source limitations. The implementation plan gives build stages, tests T01-T12, input questions Q01-Q14 and official technical references. Requirement and question IDs are stable anchors for future changes.

Suggested reading order for a first review: problem statement, handbook pages 1-3, implementation plan questions on pages 11-12, then the remaining technical sections.

02 / SYSTEM STRUCTURE

Architecture and trust boundaries

A component becomes trusted because of its deployment and verified identity, not because the agent says it is trustworthy.



Logical architecture. MVP may combine services; cloud-specific adapters remain replaceable.

Four boundaries to preserve

Human administration. Owners propose agent metadata; authorized reviewers approve changes. Operators suspend agents. Auditors inspect evidence. The portal is a client of the control API and cannot bypass its role checks.

Untrusted agent execution. Agents may receive misleading instructions from users, documents or tools. Their requests are inputs to policy, never authority to change policy. They do not hold broad downstream credentials.

Trusted enforcement. The gateway resolves authenticated identity and current task authority, evaluates rules and executes only approved operations. Its adapter service holds narrowly scoped downstream credentials.

Protected resources and evidence. Tools verify the gateway or workload identity and enforce their own resource authorization where possible. Audit writers and readers use separate access roles; evidence storage must not be editable by agents.

Why control and execution are separate

The control plane changes relatively slowly: owners, policy releases, tool registrations and approvals. Runtime enforcement happens for every protected call. Separating these responsibilities lets a central policy model serve several clusters or clouds while enforcement stays near the workload. A distributed deployment still needs fresh suspension state and consistent policy versions.

Coverage depends on integration: if an agent can directly reach an ungoverned tool, external model endpoint, filesystem or local process, this gateway cannot claim to control that path. Inventory and network restrictions therefore form part of the design, not just deployment convenience.

03 / WORKED EXAMPLE

What happens during one task

Illustrative case CASE-104 uses synthetic data. Names and identifiers below are examples, not actual records.

Step	System behavior	Visible evidence
1. Start task	A signed-in employee requests a case summary. The task service verifies their case access and creates run-104 with purpose case_assistance.	Requester reference, approved task type, run ID and grant expiry.
2. Authenticate agent	Coordinator agent A presents its workload credential. Gateway resolves active agent A, version 1 and the correct environment.	Identity mapping and registry version; secret excluded.
3. Delegate retrieval	A asks agent B to retrieve approved knowledge. Task service creates child run-105 and a narrower read grant.	Parent/child link, both identities, scope and depth.
4. Read knowledge	B calls the registered knowledge tool through the gateway. Policy allows selected article retrieval.	Decision d-201, rule version and tool outcome.
5. Read case	A requests the permitted fields for CASE-104. Gateway filters the response before returning it.	Decision d-202 plus returned-field labels and masking outcome.
6. Propose update	A delegates an eligible update to agent C. A permitted status change needs an independent reviewer.	Approval a-301 bound to C, run, exact arguments and case version.
7. Approve and execute	Reviewer approves the displayed change. Gateway rechecks policy, resource version, grant and suspension; reserves one-use execution; calls the tool.	Reviewer, execution key and confirmed outcome, all linked.
8. Finish	The coordinator summarizes the permitted results. Dashboard connects the entire run and marks any incomplete operation.	Task completion, timeline and sanitized evidence export.

The same scenario must demonstrate denial

If B requests unrelated customer records, its narrowed grant fails. If a retrieved document tells A to send data to an external website, destination policy fails. If an approval is replayed with a different status value, its request digest fails. If A is suspended, future child calls fail through their parent-grant ancestry. In each case, verify the protected tool did not execute.

What an investigator should see

Start with run-104 and inspect the requester, agent owner/version, parent-child chain, exact action summary, policy version, decision reason, approval, tool result and timestamps. The timeline records observable actions and approved context. It does not need private model chain-of-thought or full raw prompts to explain an access decision.

04 / AGENT MANAGEMENT

Identity, ownership and lifecycle

Identity establishes who is calling. Authorization establishes what that caller may do now.

Three identifiers that must remain distinct

Agent definition. A durable registration such as case-coordinator. It has an owner, purpose and versions. Ownership is accountability, not a credential.

Runtime instance. A particular deployed workload using an approved version. Its credential must map to the agent, environment and deployment binding. Several replicas may use the same logical definition.

Task/run. One execution for an approved purpose. Its grant may be narrower than the agent's general eligibility. Human-requested tasks also carry the verified requester's authority.

Transition	Required checks / effect
Draft to submitted	Required metadata, owner, purpose, tool requests, version and risk level are present.
Submitted to approved	Reviewer assesses purpose, data and access. Record who approved which immutable version.
Approved to active	Deployment identity binding and environment controls pass verification. Grant issuance may begin.
Active to suspended	Authorized operator supplies reason. Deny new grants/calls; revoke descendants and flag active external operations.
Suspended to active	Reviewer verifies remediation and current approvals. Issue fresh grants; do not revive revoked ones.
Any eligible state to retired	Disable runtime bindings and credentials; keep required history. New use requires a new approved lifecycle entry.

Credentials and grants

Credentials prove an identity to an intended recipient. A task grant describes the current authorized scope and links to stored revocation state. Short expiry limits exposure but does not provide immediate revocation on its own. Each protected dispatch must check active state and grant ancestry. When proof or current state is missing, the proposed default is denial.

Ownership changes and version changes

Reassign departing owners through an authorized workflow; record old and new responsibility and any access review. Changing an owner must not erase history. A material change to tool permissions, task purpose, model, prompt or deployment should be reviewed using the project's risk rules. Preserve the version used by past runs so the evidence remains explainable.

Suggested demo values: five-minute task grant, ten-minute approval expiry and delegation depth of two edges. These are independent controls; approval cannot revive an expired grant.

05 / DECISION MODEL

How policy and data rules work

Decisions use verified identity, trusted metadata and the concrete requested action.

A readable decision rule

Example: case information access

Allow a case read only when the agent is active, its approved purpose is case_assistance, the task grant covers this case and read operation, the human requester may access the case, the tool is registered for this environment and the response fields are permitted. Return a masking obligation for restricted identifiers. If any mandatory fact is missing or any explicit deny applies, deny.

Inputs and outputs

Input categories: authenticated subject, approved agent metadata, verified requester/service mandate, run and grant chain, registered tool, action, resource, canonical arguments, source data labels, destination, approval reference, current time and policy revision. The agent may propose arguments; it cannot supply its own trusted entitlement or classification.

Output categories: allow, deny or require_approval; decision ID; reason codes; policy version; validity bound; required obligations. Obligations are actions such as field projection, a response byte limit or exact-request approval. If an adapter cannot perform an obligation, the gateway denies the call.

Rule category	Why it exists
Identity and lifecycle	Unknown, retired or suspended agents should have no active tool authority.
Action and resource scope	An approved purpose must not become permission for arbitrary records or operations.
Delegation attenuation	Child authority stays within parent authority and the child's own eligibility.
Data and destination	Permitted access to data does not imply permission to send it anywhere.
Freshness and approval	A past approval must not authorize a changed request or survive revocation.
Rate and cost limits	A valid agent can still loop or consume excessive resources; budgets apply across its task tree.

What data controls can and cannot promise

Field allowlists and source classifications give deterministic controls for structured demo data. Text pattern detection adds useful checks but can miss transformed, encoded or inferred sensitive information. Prevent broad access and unapproved destinations first. Apply request and response checks before release; include model prompts and agent-to-agent payloads in the same design.

Rules and risk models should be understandable to reviewers. A model-generated risk explanation may assist review, but only an authorized policy release or human workflow may change access.

06 / INFORMATION MODEL

Records and relationships

This proposed logical schema makes accountability and task history queryable without storing full sensitive payloads.

Record	Key fields and relationships
Agent / AgentVersion	agent_id, tenant_id, owner_ref, purpose_id, risk_tier, state, review_due; version_id, artifact_digest, approved_tool_set and approval_ref.
DeploymentBinding	binding_id, agent/version, environment, issuer, verified workload subject and allowed runtime selectors. Unique within trust domain.
Tool / ToolVersion	tool_id, owner, endpoint_ref, auth_adapter, schema_digest, allowed_actions, data_labels, risk_level, version and environment.
TaskRun	run_id, root_run_id, parent_run_id, agent/version, requester_ref or service_mandate, purpose_id, status and trace_id.
Grant	grant_id, run_id, subject, audience, action/resource envelope, data constraints, parent_grant_id, expires_at, revocation_version and status.
PolicyRelease	policy_version, policy_digest, effective time, reviewer, test_report_ref and rollback predecessor.
Decision	decision_id, request_id, run_id, policy_version, normalized action summary, context versions, result, reasons and obligations.
Approval / Execution	approval_id, decision/run/agent, request_digest, resource_version, reviewer, expiry and state; execution_key, tool_ref and confirmed/unknown outcome.
AuditEvent / Outbox	event_id, tenant/environment, recorded_at, occurred_at, actor_ref, run/decision/execution IDs, event_type, sanitized details and delivery state.

Relationship rules

An agent has many immutable versions and deployments. A root task has zero or more child tasks; each grant belongs to one subject/run and optionally references a parent grant. A decision may generate an approval; an execution consumes the matching authorization and produces outcome events. Tenant and environment must be verified on every relationship, not just the top-level query.

Integrity, privacy and queries

Use foreign keys, state constraints and optimistic version checks. Enforce uniqueness for execution idempotency and event IDs. Index task ancestry, agent/status, grant expiry, decision time and unresolved executions. Use opaque resource references in routine logs; a simple hash of a predictable customer identifier is not sufficient anonymization.

Routine evidence holds IDs, labels, reason codes and digests. If forensic payload retention is later required, use a separately authorized, encrypted store with explicit access, retention and region rules. The admin portal must not expose credentials or raw sensitive inputs.

07 / INTERFACE DESIGN

Proposed API contracts

These endpoints describe intended boundaries. They are not implemented or ready for direct use.

Interface	Purpose and important condition
POST /v1/agents	Create draft registration. Owner role required; server assigns agent and tenant references.
POST /v1/agents/{id}/transitions	Submit, approve, activate, suspend or retire. Check role, expected record version and reason.
POST /v1/runs	Create a task and authorize its requested scope using verified requester/service authority.
POST /v1/runs/{id}/delegations	Create child run and grant. Parent is authenticated; permissions can only narrow.
POST /v1/tool-invocations	Main protected action boundary. Validate identity, grant, policy and data; execute only after obligations pass.
POST /v1/approvals/{id}/decision	Approve or reject the exact request. Authorized reviewer, expiry and separation-of-duty checks apply.
GET /v1/runs/{id}/events	Read ordered task evidence; tenant, role and data-sensitivity filters apply.
GET /v1/agents/{id}/activity	View inventory-linked decisions, last use, denials, delegations and unresolved operations.

Illustrative decision response

decision_id: d-203; **request_id:** req-88; **run_id:** run-106; **result:** require_approval; **reason_codes:** [CASE_WRITE_REVIEW]; **policy_version:** policy-7; **obligations:** [exact_request_approval, idempotent_write]; **approval_id:** a-301. The server stores the request digest and context versions; the agent cannot replace them.

Illustrative audit event

event_id: ev-410; **event_type:** tool.execution.completed; **run_id:** run-106; **parent_run_id:** run-104; **agent_id:** case-update; **tool_id:** case-service; **decision_id:** d-204; **approval_id:** a-301; **outcome:** succeeded; **data_labels:** [internal]; **policy_version:** policy-7. Include server timestamps and execution_key; omit token and raw case fields.

Error and retry behavior

Use 401 for failed authentication, 403 for denied authority, 409 for stale state/idempotency conflicts, 422 for invalid schemas, 429 for quotas and 503 for unavailable enforcement dependencies. A pending approval can return 202 with an approval reference, without executing the tool. Safe error bodies include reason_code and correlation_id. Never auto-retry a side-effecting request with an unknown target outcome.

If a payload or record changes after approval, create a new authorization/approval cycle. A retry with the same valid idempotency key returns the prior known result instead of repeating the side effect.

08 / OPERATING THE FRAMEWORK

Runbooks and troubleshooting

Proposed procedures to refine against the implemented deployment and exercise during acceptance testing.

Suspected compromised or misbehaving agent

1. Identify the agent and affected root runs from the alert. 2. Suspend the agent through the authorized control API and record the incident reference. 3. Revoke grants and descendants; signal cancellation. 4. Verify new calls are blocked and inspect in-flight or unknown external operations. 5. Preserve sanitized evidence and review affected tool records. 6. Fix the credential, artifact or policy issue and independently review reactivation. Completed side effects may need tool-specific remediation.

Policy release or rollback

Propose a versioned change with a clear reason and expected effect. Run policy unit tests and the affected end-to-end scenarios. Compare decisions in a non-enforcing shadow evaluation, with privacy controls, then obtain peer review. Activate a signed/integrity-checked release and record its version at gateways. Watch denial and failure rates. Roll back to the last verified release if needed; keep both versions in historical evidence.

Symptom	Checks and safe response
Unexpected access denial	Inspect reason code, agent state, task scope, requester access, grant ancestry, tool version and policy revision. Correct the cause; avoid blanket allow rules.
Policy / identity unavailable	Confirm service health and trust configuration. Protected calls stay denied until verification is restored. Report degraded availability separately from security denials.
Approval stays pending	Check eligible reviewers and exact-request details. Expired grant requires new authorization; do not extend authority silently.
Tool timed out after a write	Mark execution unknown. Query the tool by execution/idempotency reference; reconcile before retrying.
Missing or delayed audit events	Compare outbox, consumer offsets and target execution records. Stop protected dispatch if durable local recording is unavailable or its buffer is full.
Direct tool access succeeds	Treat as a coverage failure. Restrict network/credentials, retest bypass and withdraw claims of complete enforcement until fixed.

Routine operations and recovery

Monitor deny rate by reason, policy latency, gateway errors, unresolved writes, audit delivery lag, revocation age, tool changes, owner-review age and task budgets. Back up registry/policy/grant/approval state and rehearse restore with identity and secret configuration. After restore, verify active grants against revocation state and reconcile pending operations before resuming dispatch.

Production runbooks must name an on-call owner and escalation route. Recovery targets, retention and maintenance windows remain unresolved until the client confirms Q09-Q13.

09 / DESIGN RATIONALE

Why these choices were made

All decisions are proposed at v0.1. Record acceptance or revision when your constraints are known.

Decision	Reason and tradeoff
D01. Cloud-neutral contracts	The meeting explicitly allows multiple cloud preferences. Adapters preserve a common policy model, but each cloud still needs its own tested identity and networking integration.
D02. Enforcement gateway + network boundary	Provides a place to prevent unauthorized actions. Adds latency and a sensitive dependency; SDK-only integration offers weaker bypass protection.
D03. Deterministic authorization	Explicit rules are testable and explainable. Intent interpretation remains limited; a model may assist triage but must not grant privileges.
D04. Short-lived, narrowed task grants	Reduces standing privilege and constrains delegation. Requires grant state, ancestry checks and a revocation strategy.
D05. Exact-request approval	Prevents a benign approval from being reused for a different action. Introduces reviewer latency and careful retry handling.
D06. Durable audit separate from traces	Security evidence must not disappear due to performance trace sampling. Creates storage and retention responsibilities.
D07. Synthetic banking demo	Shows relevant business behavior without needing real customer systems. A later sandbox integration is necessary to prove enterprise fit.
D08. Small modular deployment	Keeps the first build understandable and affordable. Split services when trust boundaries, scaling or ownership justify it.

Risks that remain even with the framework

A compromised trusted gateway or identity service can undermine enforcement. A permitted action can still be wrong for business reasons. Text-based data detection is incomplete. Unmanaged agents and unintegrated local processes remain outside coverage. External tools may lack reliable cancellation or idempotency. Policy mistakes can cause either exposure or service disruption.

How to reduce those risks

Use independent policy review, narrowly scoped adapter credentials, tool-side resource checks, isolated workloads, dependency scanning, realistic negative tests and regular coverage reviews. Track uncovered paths and unknown outcomes as explicit operational items. Do not present the demo as proof of production security, measured scale or legal compliance.

Decision record format for the build: ID; date; status; context; options considered; selected option; rationale; consequences; owner; linked requirements; verification evidence; review trigger.

10 / KEEPING THE HANDBOOK CURRENT

Maintenance and glossary

The handbook should explain the system that exists, while preserving the reasoning behind changes.

Update rules for each implementation milestone

Change affected architecture, contracts and runbooks in the same work item as the implementation. Mark each capability proposed, implemented, verified or deferred. Attach the relevant version/commit, environment and test evidence when promoting to verified. Record discrepancies and unresolved assumptions rather than quietly replacing them with claims.

Milestone	Required handbook update
After your answers	Record Q01-Q14 decisions, agreed scope, owners, deadline and revised estimates.
After the first working slice	Replace proposed modules/endpoints with actual ones; document setup, configuration and the observed request flow.
After security controls	Add actual rule examples, role mapping, data labels, delegation/revocation behavior and negative-test evidence.
Before demonstration / handover	Add reproducible startup/reset steps, screenshots, deployment dependencies, known gaps, recovery procedure and acceptance results.
Before production pilot	Record environment-specific trust, identity, network, retention, support, recovery and capacity decisions.

Terms used in these documents

Term	Meaning
IAM / IdP	Identity and access management / identity provider: systems that manage identities and authenticate callers.
OIDC / OAuth	Standards used for identity federation and delegated authorization; concrete flows require compatible implementations.
MCP	Model Context Protocol: an interface through which agents can interact with tools and other capabilities.
PDP / PEP	Policy decision point evaluates rules; policy enforcement point applies the result. Here: policy engine and gateway.
Grant / attenuation	A bounded permission for a task / narrowing that permission when delegating.
DLP / egress	Data loss prevention controls / traffic or data leaving a workload boundary.
Idempotency / outbox	Avoid repeating a side effect / transactional recording for reliable later event delivery.

Revision log: v0.1, 09 October 2026 - initial design derived from the supplied image and partial transcript. All implementation and verification entries are pending.

Sources: user context S1-S2 is described in the problem statement. The implementation plan, page 12, links official MCP, IETF, SPIFFE, OPA, OpenTelemetry, FastAPI, Keycloak, Kubernetes and Docker references supporting the proposed mechanisms. Project defaults and operating procedures are recommendations.